Payobills Constitution

Core Principles

I. Monorepo with App Isolation

Each application MUST live in its own directory under `apps/`. Shared utilities and libraries MUST go in `common/`. No app may directly import another app's internal code — cross-app dependencies MUST flow through `common/`. This ensures each service can be reasoned about, deployed, and changed independently.

II. Helm-Based Deployment

Every application MUST be packaged and deployed as a Helm chart. There MUST be no ad-hoc `kubectl apply` workflows for application-level resources. Chart values MUST be the single source of truth for environment-specific configuration. This aligns with the homelab Kubernetes cluster managed by the sibling project `kube-homelab`.

III. Homelab-First, kube-homelab Dependency

The deployment target is the local Kubernetes cluster provided by `kube-homelab` (sibling project). Deployments MUST assume `kube-homelab` is operational. Infrastructure concerns (ingress, storage classes, cluster DNS) are the responsibility of `kube-homelab`, not this project. This project MUST NOT duplicate cluster-level infrastructure definitions.

IV. Test Coverage is Aspirational, Not Blocking

The project currently lacks tests in many areas. Tests SHOULD be added incrementally. Absence of tests MUST NOT block feature development or deployment. When tests are added, they MUST be meaningful and cover user-facing behavior. Test-driving new features is encouraged but not mandated at this stage.

V. Simplicity and Self-Hosted Data Ownership

Features MUST favor simplicity over cleverness. The core value proposition is self-hosted, user-owned financial data — no feature may compromise data locality or require external cloud services to function. YAGNI applies: implement what is needed now, not hypothetical future needs.

VI. HLD Design Discipline

High-Level Design (HLD) artifacts MUST describe features purely in terms of services, systems, data flows, and interactions. HLD artifacts MUST NOT contain code snippets, class names, method signatures, framework-specific constructs, or any other implementation detail. The purpose of an HLD is to reason about what systems are involved and how they interact — not how those systems are built internally. Implementation details belong exclusively in LLD artifacts, tasks, and source code.

VII. Feature Artifact Structure

Every feature specification directory MUST contain exactly the following artifacts and no others:

```
specs/<feature>/
├── checklists/
│   └── requirements.md   - spec quality checklist
├── research.md            - research findings and decisions
├── spec.md                - feature specification
├── hld.md                 - high-level design (services and systems only; see Principle
↪  VI)
├── lld.md                 - low-level design including data model
├── service-contracts.md   - API/interface contracts between services
└── plan.md                - implementation plan and task breakdown
```

No additional artifacts (e.g., `quickstart.md`, `data-model.md`, separate `contracts/` directories) MUST be created. If information does not fit one of these seven files, it belongs in the source code, a PR description, or a commit message — not a new spec artifact. This constraint keeps the spec directory scannable and prevents documentation sprawl.

Every `lld.md` MUST follow this section structure:

1. **Data Model Changes** — entity/table changes with an ER diagram (Mermaid). If no data model changes, state explicitly.
2. **DTO Changes** — input/output data transfer object changes per service affected. If no DTO changes, state explicitly.
3. **Frontend Changes** *(include only if the feature involves UI changes)* — one subsection per page that requires changes, each containing:
   - What changes on that page
   - Decision on which components to build or reuse

VIII. Diagram Standards

All diagrams in spec and design artifacts MUST be written in Mermaid syntax. Image-based or proprietary diagram formats MUST NOT be committed.

Every Mermaid diagram MUST be validated before committing using the official CLI:

```
docker run --rm -u `id -u`:`id -g` \
  -v /path/to/diagrams:/data \
  minlag/mermaid-cli -i diagram.mmd
```

Replace `/path/to/diagrams` with the directory containing the `.mmd` file and `diagram.mmd` with the filename. If Docker is not running when diagram validation is attempted, raise an error and instruct the user to start Docker before proceeding. Do not skip validation.

A document containing a diagram that fails to render MUST NOT be marked as ready for review or considered complete. Rendering failures MUST be fixed before the document can advance to the next workflow stage.

Deployment Model

All apps are deployed to a Kubernetes cluster managed by `kube-homelab`. Each app has its own Helm chart. The `common/` directory contains shared Docker base images, k8s helpers, and reusable chart templates. Deployment pipeline changes MUST be validated against the running homelab cluster before merging.

Apps: - `bills` — Core bill management service - `bills-parser` — Parses bill documents - `files` — File storage service - `ocr` — OCR processing - `payments` — Payment recording - `transaction-parser` — SLM-

powered transaction parsing (runs on local PC, not RPi) - `transaction-syncer` — Syncs transactions from external sources - `ui` — Frontend application - `user` — User management service

Development Workflow

- The project uses mainline development. All work targets `main` directly. There is no `develop` or `master` branch.
- PRs MUST be raised against `main`. Short-lived feature branches MUST be merged to `main` and deleted.
- Branches MUST be rebased against `origin/main` before merging.
- Each app is independently deployable; deploy only changed apps when possible.
- CSS MUST NOT use negative margins or negative padding — restructure layout instead.
- `.tool-versions` MUST NOT be modified; it pins the Rust toolchain for CI.

Governance

This constitution supersedes all informal project conventions. Amendments require: 1. A pull request updating this file with rationale. 2. Version increment per semantic versioning rules (MAJOR: principle removal/redefinition; MINOR: new principle/section; PATCH: clarification/wording). 3. Sync impact report updated in the HTML comment above.

All feature plans MUST include a Constitution Check gate verifying compliance with principles I–VIII before implementation begins.

Version: 1.3.2 | **Ratified**: TODO(RATIFICATION_DATE): use project start date ~2023 | **Last Amended**: 2026-06-13